RESEARCH RECORD · 2026

제조 현장을 이해하는 AI

로컬 LLM 연구 기록: 모델 비교부터 파인튜닝, RAG, 검증 구조까지

KIM MIN SU · SOFTWARE DEVELOPER / RESEARCHER

SUMMARY

모델에게 모든 것을 외우게 하는 대신, 역할을 나눴습니다

연구 문제

제조 데이터를 자연어로 조회하는 로컬 AI를 만들고자 했습니다. 온프레미스 환경에서 동작하고, 답변의 근거를 확인할 수 있어야 했습니다.

핵심 결론

파인튜닝은 말투와 형식에 효과가 있었지만, 변하는 DB의 사실을 안정적으로 기억시키지는 못했습니다. 사실은 검색으로 가져오고 실행 전에 검증하는 구조가 더 안전했습니다.

담당 범위

모델 비교, 학습 데이터 구성, 파인튜닝 실험, RAG·Text-to-SQL 구현, 평가와 원인 분석, Docker 배포와 모니터링을 수행했습니다.

보안을 위해 실제 스키마, 데이터, 내부 평가 수치와 사내 시스템 정보는 제외했습니다.

RESEARCH FLOW

실험 결과에 따라 다음 방향을 결정했습니다

01

모델 선정

조건별 비교

02

파이프라인

학습 절차 검증

03

스키마 CPT

구조 학습 시도

04

RAG 연결

검색 근거 추가

05

코더 모델

SQL 능력 비교

06

원인 분석

성능 저하 추적

07

구조 개선

검색·검증 강화

08

재학습

망각 대응

EXPERIMENT 01

조건을 정해 후보 모델을 좁혔습니다

가설

라이선스, 응답 속도, 답변 정확도를 함께 봐야 실제 도입 가능한 모델을 고를 수 있다.

방법

GPT-OSS, Qwen, Llama, EXAONE, Gemma 계열 등을 같은 질문과 조건으로 비교했습니다.

결과

기준을 동시에 충족한 후보를 추렸고, 여러 모델을 동시에 튜닝하지 않고 하나로 먼저 검증하기로 했습니다.

판단

동시에 여러 변수를 바꾸면 실패 원인을 구분할 수 없어, 가장 가볍고 빠른 모델로 파이프라인부터 검증했습니다.

EXPERIMENT 02

작은 데이터로 학습 파이프라인을 먼저 검증했습니다

가설

데이터를 키우기 전에 전처리, 학습, 평가 절차가 끝까지 작동하는지 확인해야 한다.

방법

JSONL 데이터 구성, 중복 제거, 학습·검증·시험 분리, LoRA 학습과 추론 비교를 진행했습니다.

결과

튜닝 모델은 메뉴 경로와 답변 형식을 짧고 구체적으로 표현했고, 전체 파이프라인이 정상 작동했습니다.

판단

파인튜닝은 말투와 형식을 맞추는 데 효과가 있다고 보고, 다음 단계로 DB 구조 학습을 시도했습니다.

EXPERIMENT 03

CPT로 DB 구조를 학습시켰습니다

가설

메타데이터와 프로시저 문서를 학습시키면 모델이 실제 스키마를 기억해 SQL을 작성할 수 있다.

방법

스키마와 프로시저 문서를 CPT로 학습시키고, 대표 조회 질문으로 생성된 SQL을 확인했습니다.

결과

쿼리의 구조는 일부 학습했지만, 테이블과 컬럼을 지어내거나 잘못 연결하는 문제가 남았습니다.

판단

CPT만으로는 최신 스키마와 정확한 조회를 보장할 수 없다고 판단하고, RAG로 근거를 넣는 방향으로 전환했습니다.

EXPERIMENT 04

RAG로 스키마 근거를 넣었습니다

가설

질문과 관련된 문서를 찾아 넣으면 없는 테이블을 지어내는 문제를 줄일 수 있다.

방법

스키마와 프로시저 문서를 벡터 DB에 넣고, 베이스 모델과 튜닝 모델을 같은 질문으로 비교했습니다.

결과

잘못된 이름은 줄었지만, 쓰지 않는 테이블이 검색되거나 조인 규칙이 없으면 결과가 틀렸습니다.

판단

검색 범위를 조정하고 조인 규칙을 문서화했습니다. 다만 복잡한 조건은 모델의 SQL 능력 자체가 부족하다고 판단했습니다.

EXPERIMENT 05

코더 모델과 SFT를 비교했습니다

가설

코드 작성에 강한 모델을 쓰고 SQL 예제로 학습시키면 조회 능력이 높아질 것이다.

방법

코더 모델을 베이스 모델과 비교하고, 자연어와 SQL 쌍으로 SFT를 진행한 뒤 같은 문항으로 A/B 평가했습니다.

결과

코더 베이스 모델은 SQL 작성 능력이 좋았지만, SFT 모델은 오히려 일반 추론과 되묻기 능력이 약해졌습니다.

판단

기대와 반대되는 결과였기 때문에 추가 학습을 멈추고 원인을 먼저 찾기로 했습니다.

EXPERIMENT 06

성능 저하의 원인을 추적했습니다

가설

파인튜닝 자체보다 학습 데이터와 학습 조건에 문제가 있을 수 있다.

방법

학습 로그, 손실 값, 테이블 분포, 응답 형식, 서빙 코드를 차례로 점검했습니다.

결과

과적합, 좁고 반복적인 데이터 범위, 일부 학습 데이터의 응답 형식 오류를 확인했습니다.

판단

모델을 더 학습시키는 대신, 이미 확보한 문서를 제대로 찾아 쓰는 검색 구조 개선이 더 안전하다고 판단했습니다.

EXPERIMENT 07

검색과 검증 구조를 개선했습니다

가설

좋은 근거를 찾고 실행 전에 막으면 파인튜닝 없이도 조회 품질을 높일 수 있다.

방법

프로시저 검색을 활성화하고, 테이블·뷰·프로시저 검색을 분리했습니다. 없는 컬럼을 잡는 SQL 검증과 오류 재시도도 추가했습니다.

결과

파인튜닝 없이 조회 성공률이 개선되었습니다. 반면 편중된 예시를 많이 넣는 방식은 오히려 성능을 떨어뜨렸습니다.

판단

효과가 확인된 변경만 채택하고 Few-shot 추가 방안은 기각했습니다. 평가 문항도 업무 분야별로 넓혔습니다.

EXPERIMENT 08

한국어 CPT의 망각 문제를 다뤘습니다

가설

한국어 산업 문서로 학습시키면 공개 모델의 한국어 구사력을 높일 수 있다.

방법

산업·공학 말뭉치로 CPT를 진행하고, 유창성과 도메인 지식을 나누어 평가했습니다.

결과

도메인 답변은 유지됐지만 주제 이탈, 언어 전환, 응답 미종료가 나타났습니다. 다른 언어 데이터가 없어 망각이 생긴 것으로 분석했습니다.

판단

기존 결과에 이어 붙이지 않고 베이스 모델부터 리플레이 데이터를 섞어 재학습하고, 중간 체크포인트마다 품질을 확인하도록 바꿨습니다.

FINAL STRUCTURE

학습은 표현을, 검색은 사실을 담당합니다

01

질문 분류

조회, 되묻기, 범위 밖을 구분

02

정보 검색

관련 구조와 규칙을 확인

03

SQL 생성

근거 안에서 작성

04

검증·재시도

오류를 잡고 다시 생성

05

읽기 전용 실행

결과를 설명

서비스 구현

Streamlit으로 모델과 처리 방식을 비교하는 화면을 만들고, LangGraph로 분류부터 실행까지의 흐름을 구성했습니다.

운영 환경

Docker로 모델 자동 배포 환경을 구축하고, 인프라 현황과 모델 배포 상태·버전을 확인하는 모니터링 기능을 구현했습니다.

WHAT I LEARNED

좋은 모델보다 검증 가능한 구조가 먼저였습니다

실패도 기록합니다

기대와 다른 결과가 나오면 추가 학습보다 원인 분석을 먼저 했습니다. 실패 기록이 다음 구조를 결정했습니다.

같은 조건으로 비교합니다

변경 전후를 같은 문항으로 평가해, 효과가 확인된 변경만 채택했습니다.

데이터의 품질을 먼저 봅니다

양보다 범위와 형식이 중요했고, 연관 없는 데이터가 들어가면 답변 성향과 페르소나가 쉽게 흔들렸습니다.

남은 과제

응답 속도, 복잡한 지표의 정확도, 페르소나 유지 문제를 계속 개선하고 있습니다.